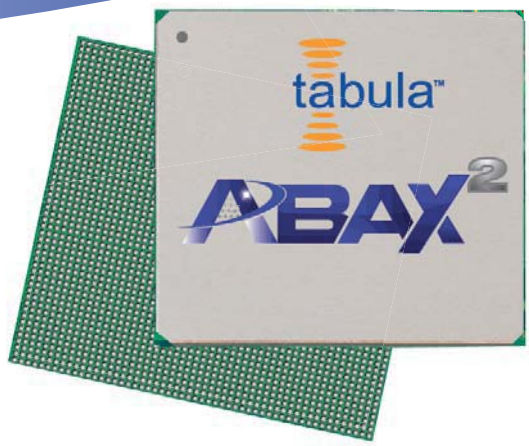
HIGHER LOGIC DENSITY AND PERFORMANCE | HIGHER MEMORY DENSITY | HIGHER MEMORY THROUGHPUT

# Challenges of High-Speed Ethernet at 100G, 400G and Beyond - The Spacetime Perspective

*White Paper*

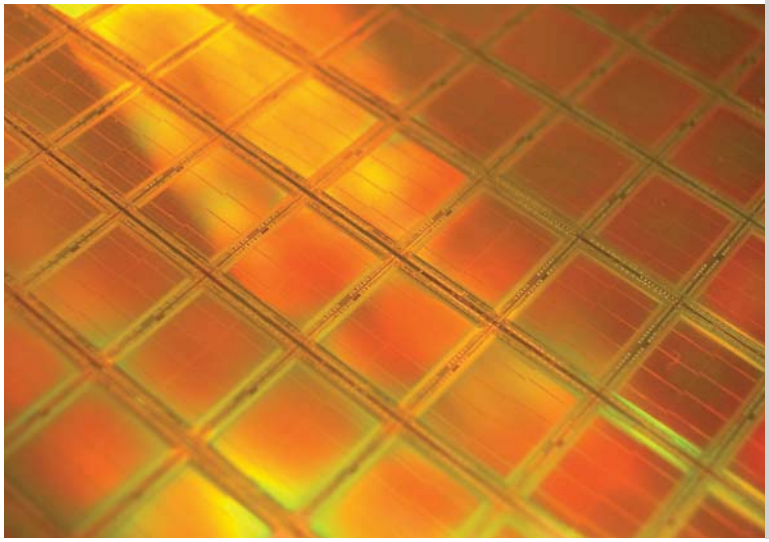





Semiconductor devices face extreme challenges in meeting the speed and throughput requirements for packet processing at 100G, 400G, and beyond. These processing challenges are compounded by an ever-changing networking landscape consisting of evolving and escalating security threats, migration from IPv4 towards IPv6 to handle IP address exhaustion, and emerging/morphing protocols such as software-defined networking. Devices that combine very high-speed network processing capability with agility are needed. Tabula is addressing this need with its ABAX$^2$ P-Series devices with the combination of their unique Spacetime® capabilities and tailored hardware features. Built using the world's most advanced transistor technology, Intel's 22 nm Tri-Gate process, and using a radically innovative 3D routing architecture, the ABAX$^2$ 3PLD was designed specifically to address silicon logic performance bottlenecks.

tabula®

Tabula, Inc.
Corporate Headquarters:
3250 Olcott St.
Santa Clara, CA 95054
Phone: 408.986.9140
Fax: 408.986.9146



## Copyrights, Trademarks and Disclaimers

©2013 Tabula, Inc. All rights reserved.

Tabula, ABAX, Spacetime, and the associated logos are trademarks or registered trademarks of Tabula, Inc. All other trademarks are property of their respective owners. All specifications subject to change without notice.

No part of this publication may be reproduced in any form (including photocopying or electronic storage on any medium) or by any means or used to make any derivative work (such as translation, transformation or adaptation) without the express written permission of Tabula, Inc.

Tabula, Inc. reserves the right to revise this documentation and to make changes in content from time to time without obligation on the part of Tabula, Inc. to provide notification of such revision or change. Tabula, Inc. provides this documentation without warranty of any kind, either implied or expressed, including but not limited to, the implied warranties of merchantability and fitness for a particular purpose. Tabula, Inc. may make improvements or changes to the product(s) and/or the program(s) described in this document at any time.

**ABOUT TABULA, INC.**

Tabula is the industry's most innovative programmable logic solutions provider, delivering breakthrough capabilities for today's most challenging systems applications. The company's ABAX2 family of general purpose 3D Programmable Logic Devices (3PLDs), based on Tabula's patented Spacetime architecture and supported by its Stylus compiler, sets a new benchmark for the capability of programmable devices at volume price points, enabling re-programmability not only in FPGA applications but also in those historically served only by ASICs or ASSPs. Headquartered in Santa Clara, California, Tabula has over 130 employees and has assembled a leadership team consisting of industry veterans and successful entrepreneurs. Tabula is backed by top-tier investors with a long-term view toward enduring market leadership. For more information, please visit the Tabula website at www.tabula.com




## Introduction

Semiconductor devices face extreme challenges in meeting the speed and throughput requirements for packet processing at 100G, 400G, and beyond. These processing challenges are compounded by an ever-changing networking landscape consisting of evolving and escalating security threats, migration from IPv4 towards IPv6 to handle IP address exhaustion, and emerging/morphing protocols such as software-defined networking. Devices that combine very high-speed network processing capability with agility are needed. Tabula is addressing this need with its ABAX®2 P-Series devices with the combination of their unique Spacetime® capabilities and tailored hardware features. Built using the world's most advanced transistor technology, Intel's 22nm Tri-Gate process, and using a radically innovative 3D routing architecture, the ABAX 2 3PLD was designed specifically to address silicon logic performance bottlenecks.

Packet rates at 100G push against the limits of memory devices - be they DRAM, SRAM, or embedded memory - with packet arrival times becoming much shorter than external memory latency and access times. Now, more than ever, packet storage plus buffer and queue management schemes - key design elements in an Ethernet solution - differentiate network equipment products. Well-designed products will have high throughput and low packet loss during times of congestion. Poorly designed products can fail under certain stress conditions leading to high packet loss.

Packet buffering design starts with the memory storage scheme, leading to several concerns:

- Should a design use lower cost DDR3 DRAM, low latency RLDRAM3 or internal embedded memory?
- What type of memory speedup should be used?
- How fast should the memory I/O run?
- How wide should the memory bus be?
- How much memory storage capacity is required?
- What does one do when complete packets arrive faster than the time it takes to store/read packets to/from memory?

The design of logic to manage the various buffers and queues used to store packet data and enable a guaranteed quality-of-service (QoS) level also raises many questions:

- How many queues are required? How large should each queue be?
- Should each queue be statically partitioned in memory, which is simpler to design but demands more memory than is necessary? Or should the queues be dynamically assigned, requiring less overall memory, resulting in efficient allocation but requiring more complex design practices, such as pointer processing and perhaps high-speed SRAMs?
- Should the queuing scheme be output queuing (OQ), virtual output queuing (VOQ) or combined input and output queuing (CIOQ)?
- Depending on the queuing scheme selected, what type of switch fabric speedup is required?
- How will the queues be scheduled? Should the scheduler use weighted round-robin (WRR), deficit-weight round-robin (DWRR) or weighted fair queuing (WFQ)?
- How does the design handle queue congestion, yet maintain the highest possible throughput?
- When does a packet get dropped, and which one will be dropped?



Even if the design can handle 100G line rates, poor queuing design practices can result in head-of-line blocking or non-optimal system scheduling, causing the idling of an output port. This situation could occur even though there are packets in the queue destined for that output port, thus yielding lower system throughput.

With these critical, yet value-adding features to implement, designers are still tasked with constructing a complete solution. Instead of being burdened with creating standard building blocks such as a well-defined MAC layer - which is important but not differentiating - designers can benefit greatly by using a suite of fundamental, high-performance building blocks such as network search engines (NSE), classification engines, crossbar switches, DDR3 memory controllers and Ethernet MACs. System designers are best served investing their efforts in developing their own value-adding IP to achieve the very best possible design - often a very difficult task in itself. These designers seek platforms that combine an abundance of high-performance building blocks, specifically built to facilitate 100G data rates, but also deliver the programmability and flexibility to incorporate the designer's IP, handle specification churn, and provide the ability to support feature-adds, bug fixes and optimization opportunities.

This white paper describes the key elements required for designing high-performance Ethernet packet processing solutions based on current best practices. It also highlights areas that begin to break down at line rates of 100G, 400G and beyond. Next, it describes some key high-performance IP cores that designers can leverage to build their systems, helping off-load their development work while they focus on their value-adding IP - that often being the buffer/queue management and scheduleir logic.

## Industry Overview

There is an ever increasing demand for bandwidth on Ethernet networks, driven by smartphones, tablets and notebooks. This trend is fueled by the wide adoption of social networks such as Facebook, Twitter, Instagram, and Pinterest. Wireless technology advances - such as 4G LTE with cellular rates up to 12 Mbps, and Wi-Fi 802.11n with data rates of up to 600 Mbps - are being adopted *en masse* as consumers enthusiastically upgrade their smartphones and tablets. Video and music over YouTube, iTunes, Pandora, and iHeart add to the surge.

Another revolution is taking place in the data center itself, and emanating outward. Business, scholastic and personal data, combined with their relevant apps, are moving to the cloud. Businesses are driving the multiplication of data centers with the move by numerous IT departments throughout the world to cloud computing on platforms such as Amazon AWS, Rackspace, Google App Engine, Microsoft Windows Azure, and others. Google, Facebook and their ecosystem partners are re-writing the rule book on switches and servers in the data center and beyond its boundaries.

**Table 1:** ***Data Center Trends***

| Traditional Data Center | Vision |
|---|---|
| 2 CPU chips per motherboard | 24, 48 or more CPUs per drawer |
| 1G LAN on motherboard | 10G going to 40G and beyond with silicon photonics |
| Layer 2 or 2.5 switching | Software-defined networking (SDN) |
| Enterprise separate from cloud | Hybrid model |
| Security at gateway | Security at server |



Accompanying this increase in the number of CPUs, cores, and threads in the datacenter, one also witnesses a ramping adoption of virtualization - once again driving the need for more flexible bandwidth allocation and a nimble underlying platform.

There are extreme challenges in meeting the needs of extreme-speed Ethernet processing. This processing challenge is compounded by emerging standards and ever-escalating security threats. Things are changing, and they are changing fast with no slowdown in sight. Devices that combine very high-speed network processing capability with agility are needed. Tabula is addressing this problem with its latest family of 3PLD devices based on the unique Spacetime technology.

## Design Bottlenecks

With each new generation of Ethernet standard, going from 100G to 1 Terabit Ethernet, line rates increase, packet rates increase, and the time between packet arrivals shrinks. Unfortunately, not everything scales according to Moore's Law. At a certain point in the Ethernet progression, some traditional technologies commonly used to implement networking products will hit a performance wall, and not meet the demands required for the next-generation of products.

### *External DDR3 Memory Access*

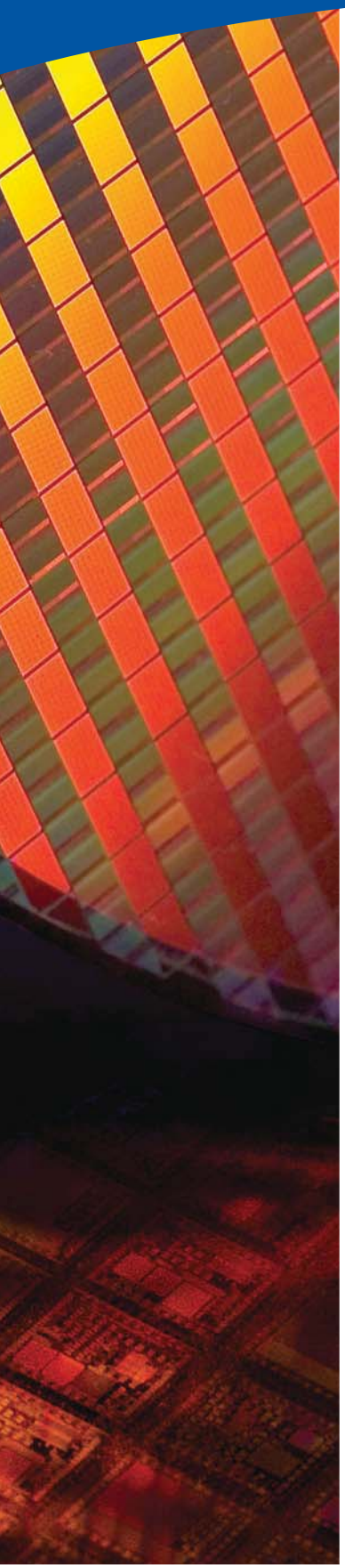
As highlighted in **Table 2**, the sheer amount of packet buffering required for some applications mandates the use of external memory storage. For memory device types, the system designer choices include RLDRAM and DDR DRAM. RLDRAM has lower latency, but generally has availability and component supply issues due to lower demand for those devices. DDR3 DRAM is abundant; however, they are designed primarily for the PC and server market - both applications that are not as sensitive to latency. Given the two options, DDR3 DRAM continues to be a popular choice for networking products, due to its low cost and high availability.

**Table 2:** ***Buffer Storage Requirements per Application***

| Application | RTT | 100G | 400G | 1 Tbps |
|---|---|---|---|---|
| Data Center Network | 250µs | 3.1 MB | 12.5 MB | 31.25 MB |
| Service Provider Metro Network | 40 ms | 500 MB | 2 GB | 5 GB |

Using DDR3 DRAM to handle packet buffering is quite complex and requires careful engineering, especially when data rates approach 100G. Currently, the minimum access time is 46.875ns for DDR3-2133 - the fastest speed grade in the DDR3 family. But for 100G, packets can arrive every 6.72ns when minimum 64-byte packets are received at full line rate. As a result, seven packets can arrive before the first packet can even be stored in memory, resulting in lost packets if the DRAM cannot keep up with this burst of data. Streaming in 64-byte packets at full line rate is a common killer stress test to check how well a switch, or other network device is designed.

In addition, for those designs that must use external DDR3 memory, 2x memory overspeed may be required to handle, for instance, the 65-byte packing problem. Due to the inefficient packing of 65 bytes on the internal data bus, two memory accesses are required for each 65-byte packet (compared to just one memory access required for 64-byte packets). To properly handle 65-byte packets running at full line rate, 2x DDR3 memory overspeed is required. For 100G, the required bandwidth of the DRAM interface for 65-byte packets, with appropriate speedup, is 400G (200G write, 200G read), thus requiring a minimum 188 data lines to the memory if using the fastest DDR3-2133 device. Streaming 65-byte packets at full line rate (100G) is a common killer stress test to check how well a switch or other network device is designed.



### *Logic Speed and Semiconductor Capabilities*

FPGAs are often used for packet processing at 100G and above when ASICs are not a viable option. System performance is approaching physical limits with each new generation of FPGA. As transistor geometries shrink, interconnect delay relative to logic delay increases. As a result, interconnect delay dominates the total path delay in a given clock cycle. Maximum achievable system clock rates on the order of 300 to 400 MHz can be expected on today's FPGA platforms.

As a result of this FPGA performance limitation, very wide data buses must be used to maintain throughput for high-data-rate applications. Using bus widths on the order of 500 to 1,000 bits is not uncommon for 100G applications. For 400G and beyond, the only solution would be more extreme bus sizes on the order of 1,000 to 2,000 bits, producing diminishing returns as achievable clock rates decrease due to timing closure difficulties in routin these enormous bus connections.

Massively wide buses have a second negative performance impact. With the minimum Ethernet packet size being 64-bytes (512 bits), multiple packets can fit within the wide bus. For instance, if one uses a 2048-bit bus to implement 400G, four minimum-sized packets can fit within the bus. Very complex and/or duplicate logic would be required to handle the various corner-case scenarios where the bus can encompass various combinations of one, two, or three complete and/or partial packets, as well as the case where the bus contains four complete packets. In addition, duplicate logic and/or extra internal speedup would be required to handle the simultaneous processing of parallel packets on the wide bus. For example, with a wide bus encompassing four packets, four memory accesses may be required on the single bus' clock cycle to store each respective packet, thus requiring either four separate identical memory controllers, or one single memory controller with a 4x speedup. Such increases in logic duplication and complexity place additional burdens on routing and timing closure, resulting in a lower than expected achievable system clock frequency.

Additionally, FPGAs placed and routed to run above 300 MHz suffer from drastically diminished utilization of their logic resources.

## Leading-Edge Technology: Spacetime Technology and ABAX² 3PLDs

With current solutions running up against a performance wall, the challenges to designing digital logic solutions to handle and process data rates of 100G, 400G and beyond may be so daunting that perhaps a fundamental change in technology is required. For DRAM, that revolutionary new technology may be hybrid memory cube (HMC), where high-speed logic is combined with a stackable memory die in a 3D architecture, enabling a significant memory speed improvement over DDR3 memory.

For digital silicon, the revolutionary innovation may be Spacetime technology (as embodied in Tabula's flagship ABAX² 3PLDs), where time is used as a routing resource (3D routing), enabling the creation of a more complete routing environment whereby slower horizontal interconnect wires are used less and faster vertical (in Z or time) routing is used more frequently. Moving signals in 'Z' allows for shorter timing paths between logic elements, resulting in extremely high performance - performance well-suited for 100G data rates and beyond.

The Spacetime 3D architecture employs time, rather than space, as a third dimension, rapidly reprogramming every resource on the device to perform multiple, different functions per user cycle - up to 12 times in the ABAX² family.

Devices based on Spacetime technology, called 3PLDs, are presented as having three spatial dimensions with all of their resources distributed across 12 floors, or *folds*, which dramatically reduces die size compared to FPGAs. In addition, all components in a 3PLD - logic, RAM, multiply/accumulate blocks, and interconnect - operate in concert at the same high clock rate made possible by the shorter interconnect enabled by Spacetime, eliminating the performance bottlenecks that exist in FPGAs. The maximum Spacetime clock is 2 GHz in ABAX² devices.

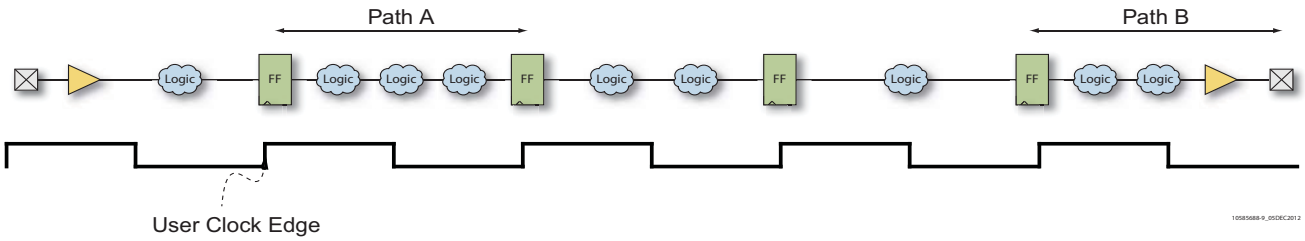
The benefits of Spacetime technology go beyond creating a faster programmable logic fabric and efficient use of logic resources. The application of Spacetime technology to certain key design elements, such as very dense embedded memory, enables powerful high-data rate capabilities. In fact, the utility of such Spacetime memory goes far beyond storing high-speed data. New processing and computational capabilities open up, which are not available with other memory structures. These computational capabilities facilitate the elegant, efficient creation of complex features and functions. As detailed below, a system designer can take advantage of these capabilities by simply dropping the blocks into their design.

### *A Performance-Optimized Architecture*

Spacetime technology uses time as a spatial third dimension, thereby offering three times the interconnect found on FPGAs, which are based on a planar logic fabric. This increase in interconnect options yields designs with a shorter interconnect and faster performance.

Another benefit of Spacetime, and its 3D routing, is the ability to implement sequential timing. In a traditional synchronous design flow, flip-flops are used to capture and propagate state information. As a result, the designer's key objective is to make the timing path between flip-flop elements, which is the aggregate sum of logic element delays and routing delays, not exceed a single user clock period. **Figure 1** depicts the timing requirement for a traditional synchronous design implemented in an FPGA.

**Figure 1:** ***Traditional Synchronous Design***




In a 3PLD, routing in the third dimension (time) is not bound by spatial limitations required by having flip-flops regularly placed on a 2D grid. In other words, state elements (flip-flops in an FPGA) can be placed at any point in time (**Figure 2**). No longer are state elements statically fixed in space, thus providing a more flexible approach to timing closure. Sequential timing uses this characteristic of Spacetime to consider timing along a full path, from end-point to end-point (and often I/O to I/O), to decide where to place the (time) storage elements in order to create enough space (i.e. time) for each element to do its work. In Spacetime technology, with sequential timing, only bandwidth and latency matter as storage elements can be

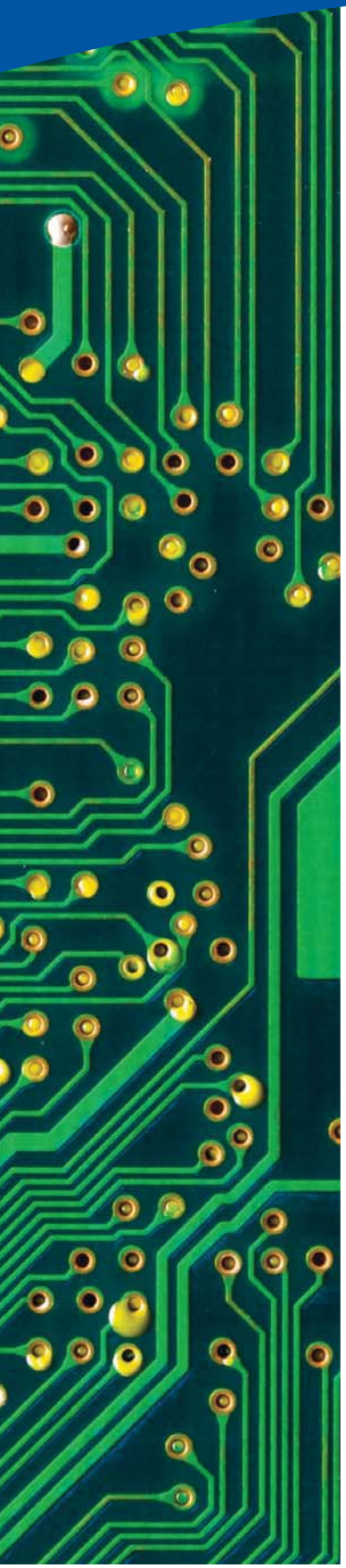
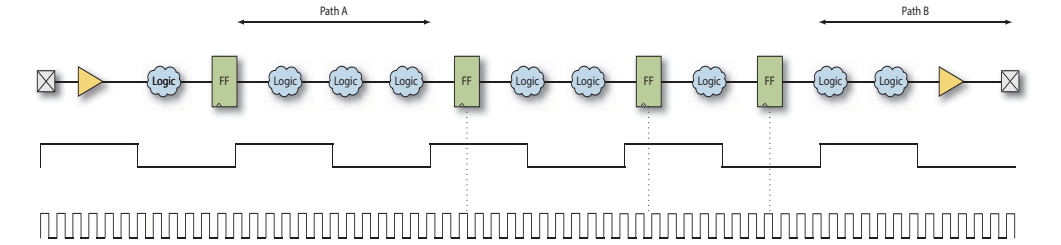


**Figure 2:** ***Sequential Timing on a Spacetime Device***




With the highly innovative 3D approach to circuit routing technology, with the smaller physical die containing shorter overall interconnects, and with the ability to efficiently allocate and meet timing paths, Spacetime technology enables higher overall system clock speeds. For high data-rate networking applications, Spacetime technology means narrower data buses can be used, narrower than what is achievable today on FPGAs. For example, for 100G data rates, use of the 256-bit data path running at 471 MHz system clock rate is achievable push-button on an ABAX$^2$ 3PLD, giving the design plenty of headroom (120G) and timing margin. With FPGAs, bus widths on the order of 512 to 1024 bits for the data path are required to achieve the same line rate, resulting in difficult timing closure due to the complexity of the design and the massive routing congestion created by the sheer number of very wide buses. For 400G data rates, use of a 512-bit data path - an ideal size if the minimum packet size is 64 bytes - is practical with Spacetime technology, rather than a bus width in the range of 2,048 bits using FPGAs. Allowing a narrower bus width eliminates the need for complex logic required when a packet partially fits within a very wide data bus, requiring multiple corner case conditions to be resolved as described earlier. By enabling narrow data buses, Spacetime technology with it 3D routing has the additional benefits of simplifying the overall design and easing timing closure.

### *Abundance of Integrated 10/40/100G EMACs*

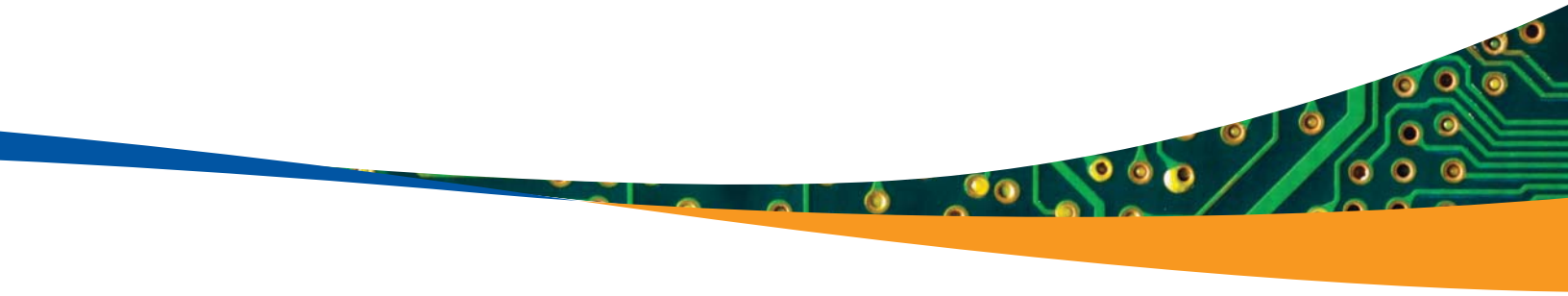
Tabula offers purpose-built circuitry implementing the complete 802.3 MAC layer, capable of operating at data rates of 10, 40, and 100G. Each ABAX2 3PLD contains an abundance of these embedded hard EMACs:

- Four 100G MACs (compliant with the finalized IEEE 802.3ba specification)
- Sixteen 40G MACs, each configurable as four 10G MACs (for a total of 64) with support for 10/100/1000 Tri-mode Ethernet

Depending on a design's needs, a combination of 100, 40 or 10G EMACs can be created. For example, a single device can support the combined operation of thirty-two 10G EMACs, two 40G EMACs, and two 100G EMACs.

These hard EMACs are self-contained drop-in design units that are easily instantiated into a system designer's code - saving the extra coding, verification and debugging effort. The use of hard EMACs frees up logic resources, allowing the designer both time and device resources to implement higher value-add features such as advanced packet buffering schemes, Carrier Ethernet, Layer 3 and above processing, among others.

In addition to the hard EMACs, the ABAX2 3PLD contains 64 high-speed serial transceivers, each capable of operating from 1.0 to 14.1G, and interfacing seamlessly with the EMACs for a complete MAC-PCS-PMA drop-in implementation. All that is required for the complete Layer2/Layer1 MAC/PHY hardware solution is to add a pluggable copper or fiber optive transceiver module.



### *Integrated Ultra-Fast DDR3 Hard Memory Controllers*

Similar to the hard EMACs, the ABAX$^2$ 3PLD contains purpose-built circuitry implementing DDR3 DRAM controllers operating up to the leading-edge rate of 2,133 MT/s. As with the hard EMACs, the use of these optimized modules frees the logic engineer from designing what can be a tricky and complex memory interface, especially at the highest rates beyond 2GT/s per memory pin. In addition to freeing up design time, these hard controllers also free up valuable logic resources.

Some high-speed networking designs require interfacing to external DDR3 DRAM, for example, a carrier-grade MPLS switch supporting millions of fine-grained connections and containing several QoS priority levels. The sheer amount of required packet buffering needed to ensure adequate guaranteed QoS levels dictates the use of external DDR3 memory. As described earlier, designs using external DRAM for packet buffering are quite complex and require careful engineering when data rates approach 100G and beyond.

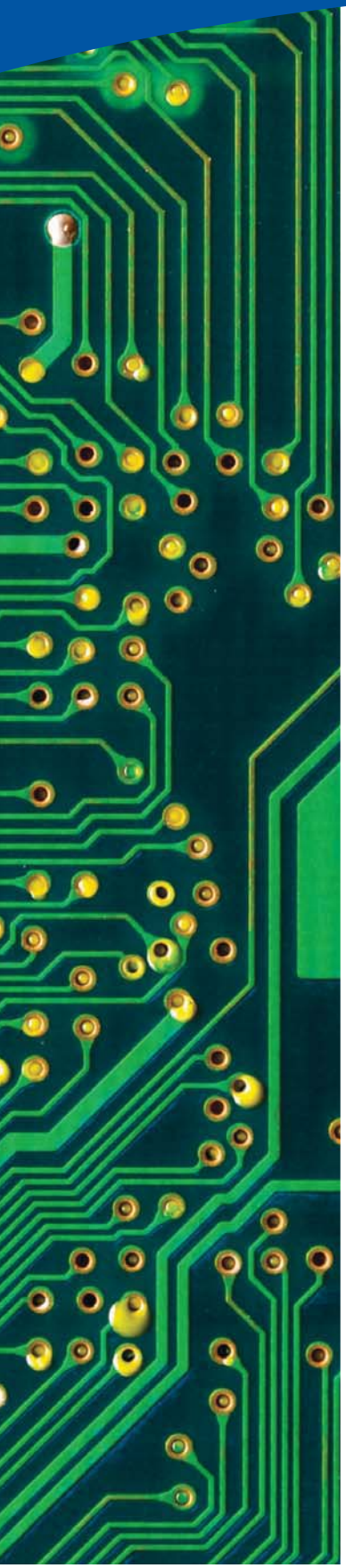
ABAX$^2$ 3PLDs include twelve DDR3 controllers capable of achieving 614G of external memory bandwidth. Each controller supports data rates of 2,133 MT/s (PC3-17000) and can be configured for widths of 8 to 72 bits in increments of 8 bits (larger bit widths reduce the number of usable controllers; for example, a single device can support four 72-bit DDR3 interfaces). Additional hard DDR3 memory interface configurations are described in the *ABAX$^2$ Handbook*. The 100G example above, where a 188-bit interface to DDR3-2133 memory is required running at 2,133 MT/s, can be easily implemented on an ABAX$^2$ 3PLD using four hard memory controllers, with each controller driving 48 bits, for a total of 192 memory data bits or three memory controllers configured for 64/72 width.

The ABAX$^2$ 3PLD is capable of supporting the fastest state-of-the-art DDR3 memories available today, and handling packet data rates running at full 100G line rates with the appropriate memory overspeed, freeing the system designer to focus on more important tasks than implementing a very difficult interface.

### *Embedded Ultra-High-Speed Memory - the Computation Story*

In contrast to high-speed transport designs - where the sheer amount of buffering mandates the use of an external memory - some applications, such as a top-of-the-rack (ToR) switch in a data center environment, open up the possibility of using embedded memory. These switches support fewer flows, have fewer queues to manage, and have shorter flow control turn around times, making it possible - and highly desirable - to handle the entire packet buffering using internal embedded memory. Using such memory blocks provides several benefits:

- Simplifies the design by removing DRAM-specific constraints such as refresh interval requirements, RAS-to-CAS delay requirements, same bank access restrictions, and precharge requirements.
- Vastly improves memory access time and memory latency. The best access time for the fastest DDR3-2133 device is ~47 ns, versus 500 ps for ABAX$^2$ embedded RAMs.
- Eliminates high-voltage I/O driving the external memory interface and their associated termination resistors, thus saving power.



This type of application requires the design platform to have a vast amount of fast internal memory, an ABAX² 3PLDs deliver. The ABAX²2P1 contains the largest amount of embedded memory in the programmable logic industry: 23.3 MB - significantly more than alternative solutions. See **Table 3** for a comparison of ABAX² memory capacity versus other alternative devices.

**Table 3:** ***Embedded Memory Comparison***

| | FPGA Vendor 1 | FPGA Vendor 2 | Tabula ABAX² 3PLD |
|---|---|---|---|
| Maximum capacity per block memory | 20kb | 36kb | 156kb for LRAM<br>18kb for MRAM |
| Addressable bytes per block memory | 2k | 4k | 16k |
| Maximum storage capacity per device | 6.5 MB | 8.5 MB | 23.3 MB |
| Multi-port capability | 2 | 2 | 12 for LRAM<br>24 for MRAM |
| Maximum memory clock frequency | 600 MHz | 600 MHz | 2 GHz |

In order to fully appreciate the impact of the ABAX² memory, one should consider how FPGAs construct their embedded memories. FPGA memories (block RAMs) are built using dual-clock RAM cells. This choice of memory cell was made so that networking applications, requiring simultaneous memory read and write operations, could support functions such as FIFOs and queuing buffers. But dual-clock memories are inherently larger blocks, taking up more physical die space and limiting the overall amount of internal memory that can fit into an FPGA.

ABAX² 3PLDs take a different approach; one more suitable for networking applications. Instead of the larger dual-clock memory cells, it uses much smaller single-clock memory cells. With this architectural advantage, Tabula is able to fit more memory storage per block and more memory storage overall per die. Each LRAM memory contains 156 Kb of memory storage, compared with 20 to 36 Kb per block on other programmable logic devices.

FPGA vendors go through great effort to implement the larger dual-clock memory cells to construct their block memories. How does the ABAX² 3PLD implement the dual-port feature needed by networking applications using smaller single-port memory cells? The answer is Spacetime technology. Similar to the logic and routing resources, the folded architecture enables memory resources to be accessed up to twelve times per user clock cycle. For the very dense LRAM block memories, constructed out of smaller single-clock cells, the memory can be accessed up to twelve times in one user clock cycle. For the smaller MRAM block memories, constructed out of dual-clock cells, the memory can be accessed up to twenty-four times per user clock cycle. Thus, Spacetime memories can easily handle the single-write/single-read operation over one user clock cycle required for dual-port memory. Additionally, multiple marallel read or write operations can be made. For example, one could implement four simultaneous writes and eight simultaneous reads to/from memory within one user clock cycle.

The ability to access memory blocks multiple times per user cycle enables faster look-up capability for faster look-up capability, translating to lower packet processing latency.

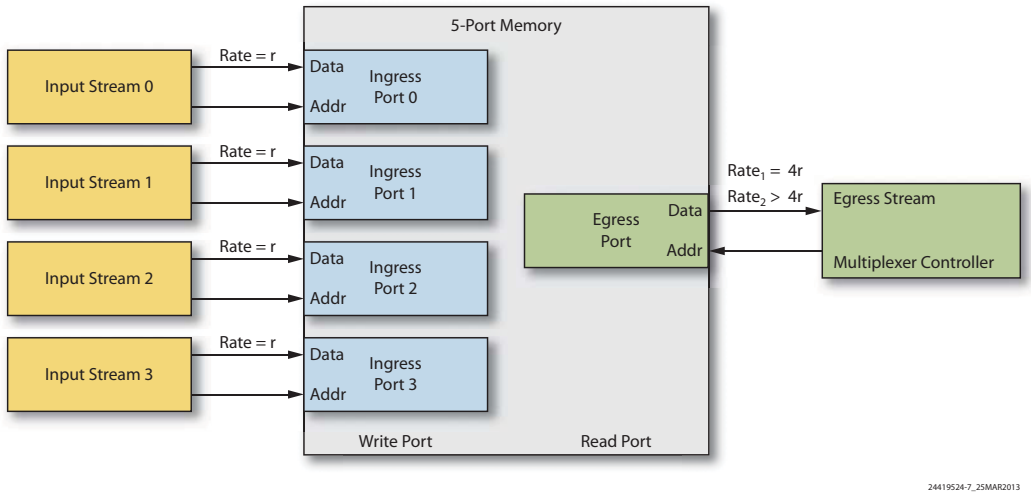
**Table 4:** ***LRAM and MRAM Memory Specification***

| Specification / Mode | LRAM | | MRAM |
|---|---|---|---|
| | Data | ECC | Data |
| Memory Size (bits) | 156K (159,744 bits) | | 18K (18,432 bits) |
| Configurations (bit width x depth) | 36b x 4K<br>18b x 8K<br>9b x 16K<br>4b x 32K<br>2b x 64K | 39b x 4K | 36b x 512<br>18b x 1K<br>9b x 2K<br>4b x 4K<br>2b x 8K |
| Write Access Widths (bits) | 36, 18, 9, 4, 2 | 39 | 36, 18, 9, 4, 2 |
| Read Access Widths (bits) | 36, 18, 9 | 39 | 36, 18, 9 |
| Ports | Up to 12 | | Up to 24 |
| Per-Fold Access | Yes | | Yes |
| FIFO Controller (optional | No | | Yes |
| Special Configuration | Cascade for increased width or depth | | Cascade for increased width or depth |
| Power-On State | User defined | | User defined |
| Maximum Unrelated Clock Domains | 1 | | 2 |
| ECC | No | Yes | No |

Another useful feature of multi-port Spacetime memories is the ability to implement multiplexing and de-multiplexing operations compactly and elegantly. **Figure 3** shows a 4:1 multiplexer implemented using 5-port memory. Incoming data is received at four ingress ports and outgoing data is sent to the egress port, at a data rate $Rate_1$ that is four times the ingress line rate (or at $Rate_2$ that is greater than four times the ingress line rate, if multiplexer overspeed is required). Each ingress line writes data to its respective memory address space and a multiplexer controller selects the ingress port (read address) to forward data to the egress port. Using a Spacetime *foldedness* of 8, data is written to memory on the first 4 folds (corresponding to the four input lines) and data is read out of memory in four subsequent folds (at an effective higher egress clock rate). If multiplexer speedup is required, such as to insert additional control or metadata in the egress stream, four additional folds can be used on the egress side for an overall 2x multiplexer speedup (using Spacetime foldedness of 12 with 4 folds for ingress, plus 8 folds for egress).



**Figure 4:** ***Spacetime 4:1 Multiplexer***




More powerful structures can be realized by combining the multi-port capability with the very deep memory storage capacity of the Spacetime memories. The multiplexible structure described above can be extended to incorporate some type of input queuing structure (**Figure 4**). The diagram shows a simple input queuing (IQ) structure, but more elaborate and sophisticated queuing, such as virtual output queuing (VOQ) with various priority levels, can be implemented as well, without loss of generality. In contrast to the multiplexer shown in **Figure 4**, the ingress data is not written to a static address location (it is not a simple store-and-forward operation). Rather, the ingress data is written to a much deeper dynamic queue. Conversely, on the egress side, data is not simply read from the memory in a simple static TDM fashion. Instead, data is read out dynamically using various sophisticated work-conserving schedulers, such as deficient round-robin (DRR) or weighted fair queuing (WFQ).

The example in **Figure 3** adds more data rates at the egress port. The case where $Rate_3$ is less than the sum of all ingress rates represents a typical statistical multiplexer with oversubscription and statistical multiplexing gain. The case where $Rate_4$ is equal to the individual ingress port rate is a special configuration that forms the basis of a Tabula crossbar switch (see "**4 x 100G Crossbar Switch**" on page 23).

**Figure 5:** ***Generic Finite State Machine***

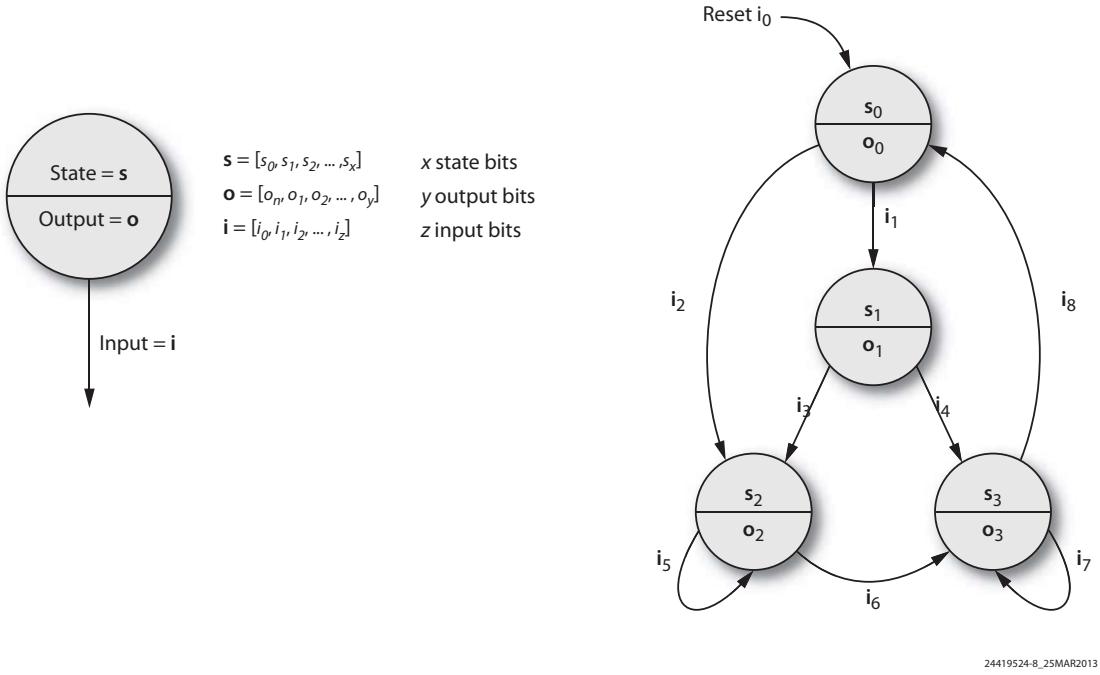
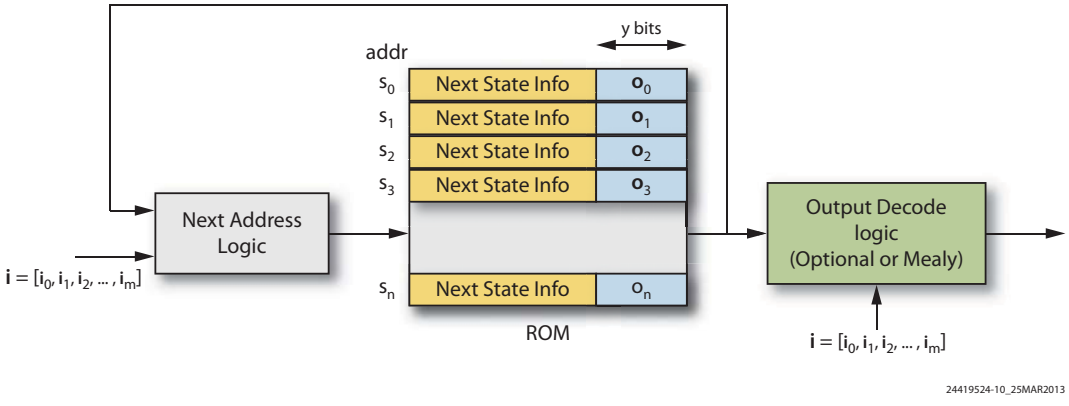



**Figure 6:** ***Spacetime High-Speed Controller***




To summarize, Spacetime memories provide a uniquely powerful design macro. With the most memory storage capacity per block and per chip in the industry, these memories can be used for typical storage operations, including on-chip packet buffering. This characteristic is highly desirable given the complexities of external storage for 100G systems. Moreover, Spacetime memories can be configured to implement efficient data path structures such as statistical multiplexers with integrated queuing buffers. In addition, Spacetime memory can be used to implement ultra-fast FSMs capable of controlling high-speed data paths. The following sections describe more elegant structures built using Spacetime memories that are suitable for high-speed networking applications.



## Key Spacetime-Enabled, High-Speed Networking Functions

In order to simplify a designer's life, Tabula not only provides the devices, ABAX² 3PLDs, and the Stylus Compiler, but also offers key building blocks required for the elaboration of high-performance (beyond 100G) packet processing designs. Some of these blocks are described below. The combination of the Spacetime architecture and Stylus software allows a designer to state the design intent and see it directly manifested. This section provides three examples:

- ***"100G Parser and Classification Engine"***
- ***"100G Associative-Search with the ATSE2 and EMSE2 Cores"***
- ***"4 x 100G Crossbar Switch"***

### *100G Parser and Classification Engine*

One such building block that Tabula offers is a packet parser engine. This engine, designed, targeted and optimized for the ABAX² 3PLD, is constructed primarily with super-fast Spacetime memory blocks and is thus capable of parsing Ethernet packets at full 100G line rate, including sustained 64-byte minimum -sized packets.

In packet parsing, key information is extracted from packet header fields, such as source address, destination address, protocol type, priority ID, etc. This information is used to make downstream decisions - decisions such as where to queue, where to switch, continue to parse, and so on. A key operation to be taken is the identification of packet boundaries and locating, via offset address or other mechanisms, specific fields of interest.

The more commonly used packet header fields are listed in **Table 5**. The fields highlighted in green are mandatory fields that should be supported in an OpenFlow Flow Table (entries highlighted in blue are values used to determine the existence of some of those fields), as specified in the *OpenFlow Switch Specification Version 1.3.1*. A good packet parser has the ability to extract all the fields of interest with low latency, which could be quite challenging at line rates exceeding 100G.

**Table 5:** ***Commonly Used Packet Header Fields***

| Field | Size | Start Value | End Value | Description |
|---|---|---|---|---|
| Ethernet SRC/DST Address | 12 bytes | 0x000000000000 | 0x7FFFFFFFFFFF | Value Ethernet addresses |
| | | 0x800000000000 | 0x8FFFFFFFFFFF | Multicast |
| | | 0xFFFFFFFFFFFF | | Broadcast |
| EtherType | 2 bytes | 0x0800 | | IPv4 |
| | | 0x86DD | | IPv6 |
| | | 0x8100 | | 802.3Q |
| | | 0x9000 | | Loopback |
| | | 0x8808 | | Pause flow control |
| | | 0x8847 | | MPLS unicast |
| | | 0x8848 | | MPLS multicast |
| | | 0x8870 | | Jumbo Frames |
| | | 0x88A8 | | Provider Bridging (stacked VLANs) |
| | | 0x88ES | | MACsec |
| | | 0x88F7 | | Precision Time Control |
| | | 0x8902 | | Carrier Ethernet OAM |
| | | 0x0806 | | ARP |
| VLAN id | 12 bits | 0x000 | 0xFFF | 12K VLAN ids |



**Table 5:** ***Commonly Used Packet Header Fields (Continued)***

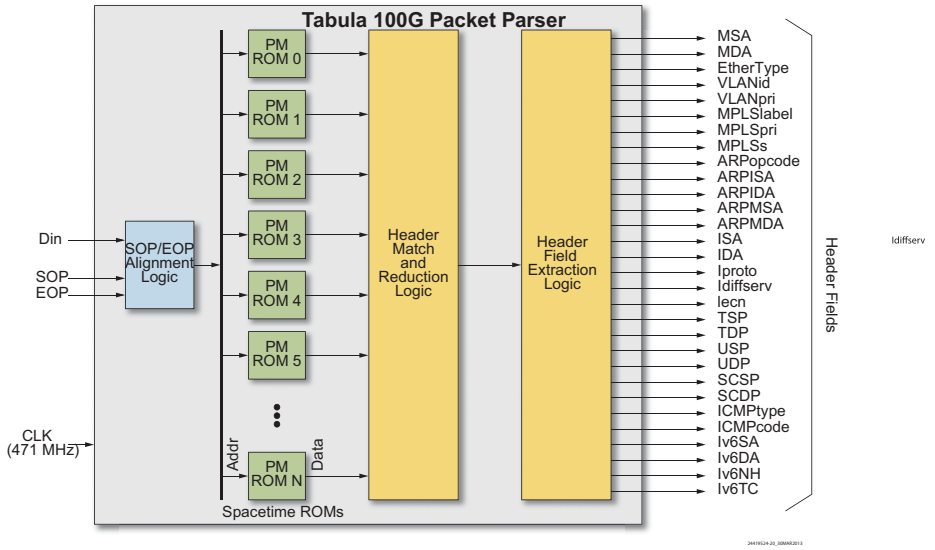
| Field | Size | Start Value | End Value | Description |
|---|---|---|---|---|
| VLAN priority | 3 bits | 000 | 111 | 8 levels of priority |
| MPLS labels | 20 bits | 0x00000 | 0xFFFFF | Valid MPLS labels |
| MPLS priority | 3 bits | 000 | 111 | 8 levels of priority |
| MPLS bottom of stack flag | 1 bit | 0 | 1 | Last MPLS label indicator |
| ARP Opcode | 2 bits | 0x0001 | 0x0002 | 0x0001 = request, 0x0002 = reply |
| ARP SRC/DST Protocol Address | 8 bytes | 0.0.0.0 | 255.255.255.255 | IPv4 addresses within ARP message |
| ARP SRC/DST HW Address | 12 bytes | 00-00-00-00-00-00 | FF-FF-FF-FF-FF-FF | MAC addresses within ARP message |
| IPv4 SRC/DST Address | 8 bytes | 0.0.0.0 | 255.255.255.255 | Valid IPv4 addresses |
| | | 10.0.0.0 | 10.255.255.255 | 24-bit block private addresses |
| | | 172.16.0.0 | 172.31.255.255.255 | 20-bit block private addresses |
| | | 192.168.0.0 | 192.168.255.255 | 16-bit block private addresses |
| | | 224.0.0.0 | 239.255.255.255 | IPv4 multicast |
| IPv4 Protocol | 1 byte | 0x01 | | ICMP |
| | | 0x02 | | IGMP (used for IPTV multicast) |
| | | 0x06 | | TCP |
| | | 0x11 | | UDP |
| | | 0x84 | | SCTP |
| IPv4 DiffServ | 6 bits | 000000 | 111111 | 64 levels of priority |
| IPv4 ECN | 2 bits | 00 | 11 | Explicit Congestion Notification |
| TCP SRC/DST ports | 4 bytes | 0x0000 | 0xFFFF | Valid TCP ports |
| TCP SYN | 1 bit | 0 | 1 | Possible firewall application |
| TCP ACK | 1 bit | 0 | 1 | Possible firewall application |
| UDP SRC/DST ports | 4 bytes | 0x0000 | 0xFFFF | Valid UDP ports |
| SCTP SRC/DST ports | 4 bytes | 0x0000 | 0xFFFF | Valid SCTP ports |
| ICMP type | 1 byte | 0x00 | 0xFF | Valid ICMP types |
| ICMP code | 1 byte | 0x00 | 0xFF | Valid ICMP code |
| IPv6 SRC/DST Address | 32 bytes | 0000:0000:0000:0000:0000:0000:0000:0000 | FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF:FFFF | Valid IPv6 addresses |
| | | 2000::/3 | | Global unicast address |
| | | FC00::/7 | | Private ULA unicast address |
| | | FE80::/10 | | Link-local unicast address |
| | | FF00::/18 | | Multicast address |
| IPv6 Next Header | 1 byte | Same values as IPv4 Protocol field | | |
| IPv6 Traffic Class | 1 byte | 0x00 | 0xFF | Valid ICMP types |



Tabula's 100G parser and classification engine building block (**Figure 7**) is specifically designed for parsing packet headers at 100G rates and beyond. It can meet such demanding performance requirements through extensive use of Spacetime memories, leveraging both their speed and unmatched storage capacity. The memories are pre-programmed with commonly used packet header field values as specified in **Table 5**. The high memory storage density enables all field values listed in the table to be stored. Also, a range of values such as valid VLAN ID range can be stored as well, giving this block great flexibility. The pre-programmed memories are then used as ultra-fast lookup ROMs. Packet parsing is accomplished by presenting incoming packet headers to a chain of Spacetime ROMs at the memory address interface. When a match is found on any bank of Spacetime memory for a particular header type - for example, EtherType-0x8100 followed by EtherType-0x0800, representing IPv4 over Ethernet on a given VLAN ID - the corresponding header fields of interest can be extracted and used for whatever specific purpose is required by the user's design. Continuing with the earlier example, a VLAN ID and its priority level can be extracted from the header along with the IPv4 source and destination address and grouped together to form a search key from which a flow ID can be extracted from an SDN search table. All this parsing at 100G data rates is doable because of the high memory density and speed capabilities of Spacetime memories.

The packet parser engine is designed to parse beginning with the Ethernet layer-2 headers. All other higher layer headers encapsulated within the Ethernet frame are extracted with respect to this Ethernet frame reference. In addition to extracting the 6-byte Ethernet source and destination address fields, the parser engine identifies and detects EtherType values and extracts all of the header fields listed in the first column of **Table 5**, taken from the higher layer headers of the header stack. These higher layer headers are layer 2.5 (MPLS, Q-in-Q VLAN), layer 3 (IP) and layer 4 (TCP, UDP, SCTP). The fields extracted from the packet parser can be used to form a search key from which flow IDs can be associated and other classification operations can be performed. In future releases, the packet parser engine will support additional fields from headers such as MACSec, IPSec and Carrier Ethernet OAM and even support higher application layer headers such as HTTP to implement deep-packet inspection features.

**Figure 7:** ***100G Parser and Classification Engine Implemented in ABAX²P1***





### *100G Associative-Search with the ATSE2 and EMSE2 Cores*

Unlike traditional RAM that reads a value from an address, content-addressable memories (CAMs) search for keys located anywhere in a table and return a value associated with that key. CAMs are a critical part of network systems as they are used to find Ethernet addresses in switches, next hops in Internet routers, flow tables for SDN and access control lists (ACLs) in firewalls.

Exact-match search engines (EMSE) find precise content in a table. For example, an EMSE can find a 48-bit key in an Ethernet MAC address table or can also identify a TCP flow in SDN by using a larger (320-bit or 640-bit) key that includes multiple fields from the header.

Ternary CAMs (TCAMs) search for content in which bits in the table take on three possible values: True (1), False (0), and Don't care (X). By allowing bits in the table to have three values, a rule can be created that matches exactly over a subset of the bits in the key. TCAMs are often used in firewalls to implement ACLs because each rule can match over a different subset of fields.

Associative search can be implemented several ways:

- Brute force - literally reading each row of the table and comparing it against the incoming key. This approach may be acceptable for very small tables, but the lookup time scales in proportion to the size of the table.
- Implementations using bit-cells that combine each bit/mask pair and key comparison circuit. This approach is fast, but is area and power intensive.
- Search engines implemented as cores that algorithmically search for keys in RAM. This structure is suitable for medium and large tables.

The Spacetime architecture is very well suited for all approaches. Regardless of the approach, the unsurpassed memory bandwidth combined with large individual memories and large total memory capacity gives ABAX$^2$ devices capabilities far above those of any other programmable devices.

Different types of packet search use different key sizes. At the lowest level, an Ethernet frame can be switched by matching the 48-bit MAC address in a table of all the hosts in a subnet. To forward packets, the lookup function needs to find at least the destination address, which is a 32-bit field in IPv4, or a 128-bit field in IPv6. Moreover, SDN often requires search across multiple fields (tuples). As a result, CAMs typically support multiple key sizes of 160-bits and 320-bits, as more bits are consumed when more fields are included in the key.

Ethernet frames contain source and destination MAC addresses. Likewise, IP packets contain both a source and destination address. UDP and TCP messages also contain source and destination port numbers that identify the application and session. As a result, packets have a minimum size that dictates the maximum rate needed by a search engine, measured as millions of searches per second (MSPS). At a line rate of 100G and a 20-byte interframe gap/preamble/Start of Frame Delimiter and 64-byte frame, a payload requires 1,011/(160+512) = 148.8 MSPS.

Tabula IP partner Algo-Logic has developed second-generation, Spacetime-optimized search engines for exact matches (EMSE2) and ternary matches (ATSE2). These IP blocks combine an algorithmic approach with extensive use of Spacetime multi-port memories, thus providing real-time, deterministic, low-latency packet processing.



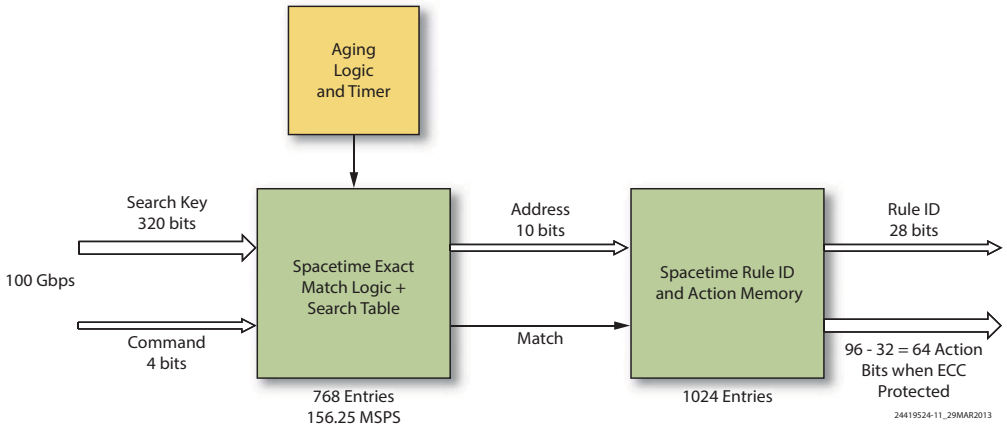
### Exact-Match Search Engine

Exact-match searching is commonly used in layer-2 switches, such as Ethernet switches, to scan multiple fields to match source and destination IP addresses, port numbers, other protocol-defined tuples and payload content. Key operations for exact-match searching in an Ethernet switch are:

1. Perform layer-2 lookups on the following fields obtained from the packet arriving on ingress port:
    a. MAC destination address
    b. MAC source address
2. For each MAC address, perform an exact-match search against the search table.
3. If a match is found for the MAC destination address, then forward the packet to the associated port or drop the packet based on an ACL rule. Reset timer for that table entry for aging. If a match is not found, then broadcast (flood)[1] the received packet to all other egress ports or drop the packet based on ACL rules).
4. If a match is found for the MAC source address, then reset timer for that table entry (for aging). If match not found, then add the new MAC address entry to the forwarding table and record the associated ingress port in search table (learning).[2]
5. Scan the entire search table. Delete any table entries that have not had any search activity after a prescribed amount of time (aging).

For a system designer, all of the operations described above (required for layer-2 switching) can be implemented simply by incorporating this single IP block (**Figure 8**) into their HDL code. All that is required is the proper programming of rules and actions for a search hit/miss, and proper assignment of the search keys. Aging is built into the block, or can be turned off if required. If IEEE 802.1 learning is required, the user can implement simple learning logic to program a new table entry upon a search miss. For an SDN switch, no additional logic is required.

**Figure 8:** ***Exact-Match Search Engine Example - 100G, 320-bit Key, 768 Table Entries***




1. *For SDN, forward the packet to the SDN controller upon a search miss rather than broadcasting it. SDN switches generally do not support IEEE 802.1D flooding.*
2. *For SDN, forward the packet to the SDN controller upon a search miss and let the controller decide how to set up the flow table, rather than implement the dynamic learning specified in IEEE 802.1D.*



The search key size shown for IPv6 is very wide at 320 bits. For layer-2 switches, normally smaller sized layer-2 header keys such as the MAC addresses, VLAN ID, or MPLS labels (for layer-2.5 switch) are applied. But IP addresses could also be used as the EMSE search key for a layer-3 switch[3] or also include the TCP/UDP/SCTP ports (the so-called 5-tuple) for a layer-4 switch. Coincidentally, a 320-bit key is an ideal size to support all the necessary Basis Class SDN header fields[4] listed in **Table 5**.

One performance parameter worth highlighting is the search rate of 150 million packets per second. For any packet processing function with Ethernet as the layer-2 tarnsport mechanism, the greatest stress test is blasting the port with minimum-sized, 64-byte packets at full line rate. Minimum-sized packets translate to the maximum packet rate offered on the line. A packet processor must process faster than this packet rate or it will fail to handle some of the packets. For 100G, the packet rate for minimum-sized 64-byte packets is 148.8 million packets per second.

The ABAX$^2$-based EMSE exact-match search engine processes at a rate of 156.25 million searches per second, giving it a 5% speedup over the worst-case 100G offered load of 148.81 million packets per second.

In addition to the high-performance capabilities, the high storage capacity of Spacetime memories make this block extremely compact. One 768-entry EMSE block utilizes 50 LRAM blocks, consuming only 4% of an ABAX$^2$P1's LRAM resources. One ABAX$^2$P1 with 10 of these EMSE instances would support 1 Tbps of exact-match search capability.

**Associative Ternary Search Engine**

The ternary search feature is commonly used for longest-prefix match operations used in layer-3 routers. Key operations of routers performing ternary search are:
1. Extract the IP destination address from a packet arriving on an ingress port.
2. Perform a value-mask result (VMR) search against the search table, with the highest priority match winning. This type of search is also known as performing a longest prefix match.
3. If a match is found, then perform action such as forwarding the packet to the next hop via associated port or drop the packet based on an ACL rule.
4. If a match is not found, then perform action such as forwarding the packet to the default route port or drop the packet based on an ACL rule.[5]

A functional diagram depicting the Algo-Logic associative ternary search engine (ATSE2) is shown in **Figure 9**. For a system designer, all of the operations required for layer-3 routing described above can be implemented simply by incorporating this single block into their HDL code. Similar to the approach taken for the EMSE2 block, the system designer just needs to provide proper search key assignment and rule ID programming for all search table entries.

***Note:*** *The search key corresponds to the IP destination address (IPv4 or IPv6) is supported. The rule ID corresponds to the port where the next hop router resides for a given longest-prefix match of the search key. A major difference for the ATSE2 is that the system designer must populate the search table with search items and mask bits.*

---

3. *A layer-3 switch is not to be confused with a layer-3 router. Layer-3 switches use exact match (with binary CAMs) for its forwarding decision, whereas layer-3 routers use longest-prefix match (with TCAMs) for its forwarding decision.*
4. *The EtherType and IPv6 next header fields are used by the packet parser to identify packets; they are not needed for search lookup. ARP opcode, ARP protocol address, and ARP hardware address may not be required for search lookup either. All other remaining SDN header fields fit within 320-bits.*
5. *For SDN, forward the packet to the SDN controller upon a search miss and let the controller decide how to set up the flow table.*



**Figure 9:** ***Associative Ternary Search Engine Example 100G, 320-bit Key, 4k Table Entries***

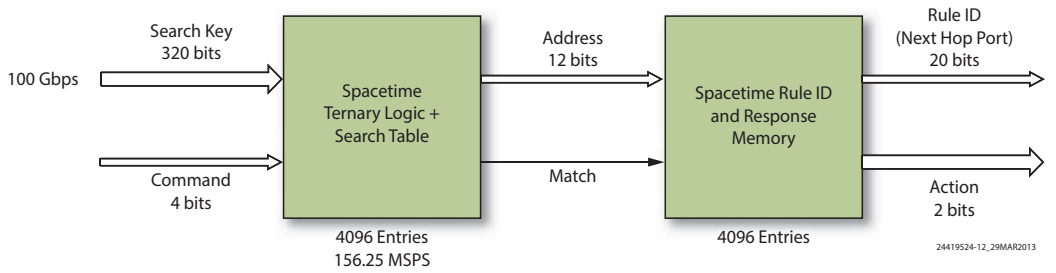
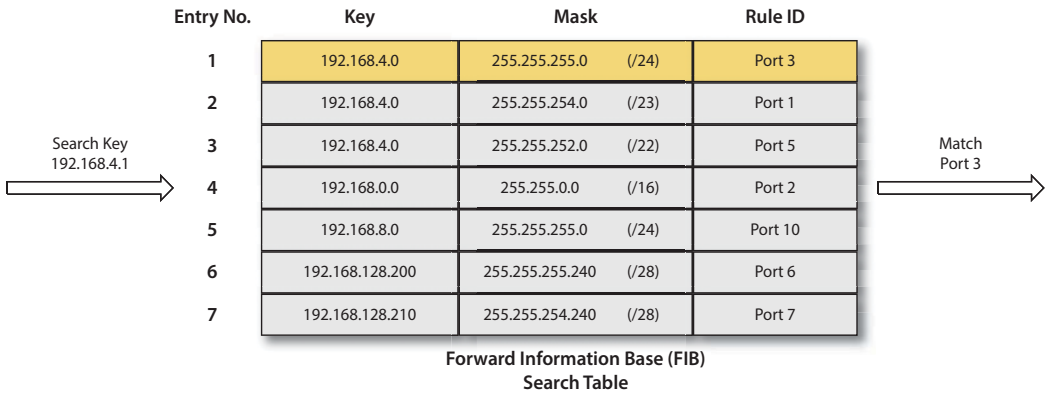



**Figure 10** depicts how a typical data plane forwarding information base (FIB) search table for a layer-3 router would be set up using the ATSE2 block. There are duplicate key values in the first three table entries but different mask values, representing different CIDR (Classless Inter-Domain Routing) blocks. An incoming search key with an IP destination address of 192.168.4.1 has a longest-prefix match with the first table entry resulting in the ingress packet being forwarded to the next-hop router located on port 3.

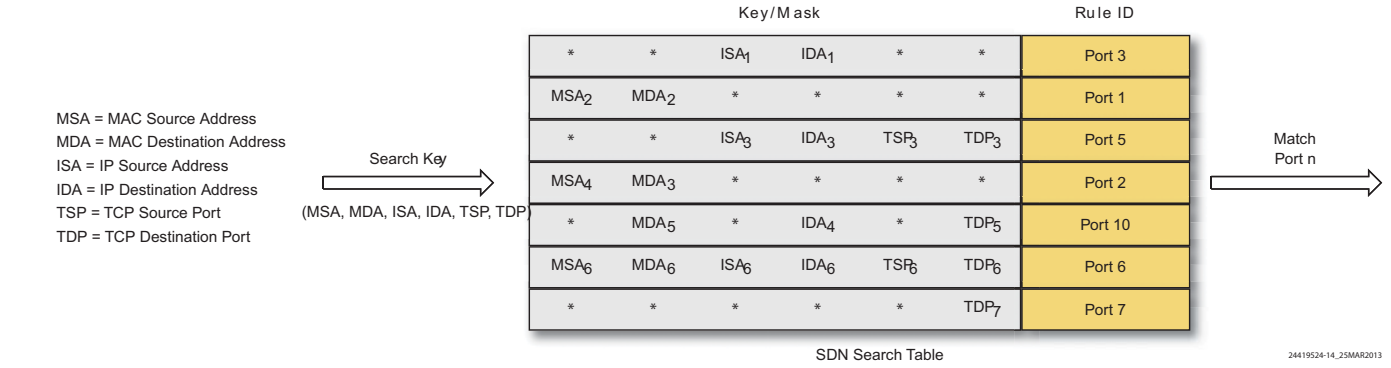
**Figure 10:** ***Layer-3 Router FIB Search Table***




| Entry No. | Key | Mask | Rule ID |
|---|---|---|---|
| 1 | 192.168.4.0 | 255.255.255.0 (/24) | Port 3 |
| 2 | 192.168.4.0 | 255.255.254.0 (/23) | Port 1 |
| 3 | 192.168.4.0 | 255.255.252.0 (/22) | Port 5 |
| 4 | 192.168.0.0 | 255.255.0.0 (/16) | Port 2 |
| 5 | 192.168.8.0 | 255.255.255.0 (/24) | Port 10 |
| 6 | 192.168.128.200 | 255.255.255.240 (/28) | Port 6 |
| 7 | 192.168.128.210 | 255.255.254.240 (/28) | Port 7 |

Not only is the ATSE2 well suited to implement typical layer-3 router operations as described above, it is also extremely useful for implementing SDN operations. SDN data-plane devices make extensive use of wildcard entries in the search table to identify various flows. A typical SDN search table implemented using the ATSE2 block is shown in **Figure 11**. For simplification, the key and mask table entries have been combined with a wildcard symbol (*) used to denote a particular field being masked out.

**Figure 11: *SDN Search Table***




The figure shows the use of a 6-tuple comprised of MAC, IP and TCP source/destination addresses and ports as the search key, but larger search key sets can be created. The ATSE2 block supports the creation of a wide 320-bit search key which is an ideal size to support all the necessary Basic Class SDN header fields listed in **Table 5**.

***Note:*** *The EtherType and IPv6 next header fields are used by the packet parser to identify packets; they are not needed for search lookup. ARP opcode, ARP protocol address and ARP hardware address may not be required for search lookup either. All other remaining SDN header fields fit within 320 bits.*

Similar to the EMSE2 block, the ATSE2 lock is comprised mainly of Spacetime memories, enabling high-performance capabilities such as those shown in **Table 6**, but differing slightly from the EMSE2.

The table below enumerates the search capabilities for instantiations of one, two, or four Algo-Logic TSE2 search cores running on the ABAX$^2$P1 3PLD.

**Table 6: *Search Engine Core Capabilities***

| Core | Table Memory | Number of Cores | Search Rate | Table Size | Item Size | Lookup Latency | DDR3 Interface |
|---|---|---|---|---|---|---|---|
| EMSE2 | On-Chip | 1 | 150 MSPS | 768 to 196K | 64-96 bits | 256 ns | - |
| | | 2 | 300 MSPS | | | | |
| | | 4 | 600 MSPS | | | | |
| | Hybrid | 1 | 150 MSPS | up to 1.5M | | 400 ns | 4 x 18 |
| | | 2 | 300 MSPS | | | | 4 x 36 |
| | DDR3 | 1 | 75 MSPS | up to 12M | | | 4 x 18 |
| | | | 150 MSPS | | | | 8 x 18 |
| ATSE2 | On-Chip | 1 | 150 MSPS | 512 to 32k | 9-12 bits | 270 ns | - |
| | | 2 | 300 MSPS | 512 to 16k(1) | | | |
| | | 4 | 600 MSPS | 512 to 8k(1) | | | |

**Notes:**
1. Lower limit for 640-bit key



### *4 x 100G Crossbar Switch*

An essential component to any high-performance networking switch, router or SDN data-plane device is the switch fabric, whether it be shared memory or crossbar switch. Tabula has devised an ingenious crossbar switch fabric, built primarily using high-speed Spacetime memories, capable of switching and forwarding 120G per port

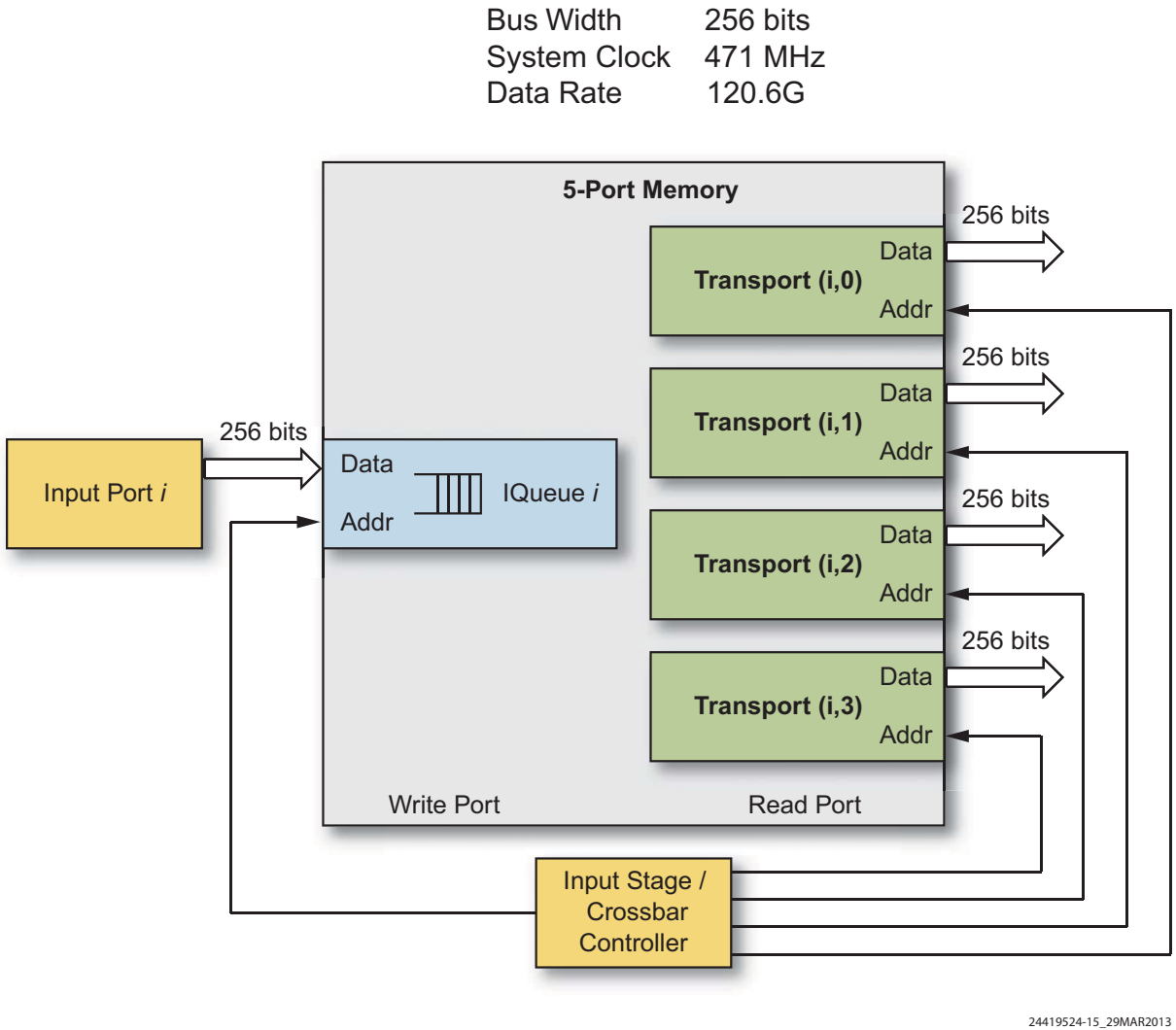
The switching elements are a variation of the Spacetime statistical multiplexer depicted (**Figure 4**). There are two elements to the switch: an input stage element (**Figure 12**) and an output stage element (**Figure 13**). The input elements can be connected to the output elements to form a 4-port 100G non-blocking crossbar switch (**Figure 14**).

***Note:*** *In* **Figure 12**, **Figure 13**, *and* **Figure 14**, *the data paths are colored blue and the control paths are colored red.*

Similar to the high-speed 4:1 multiplexer (**Figure 3**) and the statistical multiplexer, both the input and output stage crossbar elements are implemented using Spacetime memories configured as a 5-port block. Setting up a data path cross-connect is implemented by manipulating the read and write ports by a crossbar controller. The connect **Input Port 0** in **Figure 12** to **Output Port 3** in **Figure 13**, packet data is first written into the write port of **Input Stage 0**. Next, a crossbar controller specifies that the packet is directed to read port 3 (**Transport(i,3)** in **Figure 12**, where i=0). **Transport(0,3)** is is physically connected to write port 0 of **Output State 3** in **Figure 13** **(Transport(0,j)** where j=3). The crossbar controller then writes the packet to that write port, followed by a read operation at the output stage read port to complete the port 0 to port 3 cross connect. In a similar manner, a multicast from port 0 to ports 1, 2, and 3 can be achieved simply by using a Spacetime multi-port read at the input stage at read ports 1, 2, and 3, and proper co-ordination at output stage 1, 2, and 3. Thus, multicasting is inherent with this crossbar structure with no additional logic required.

What's notable in the input and output stage elements is the ability to use the deep memory storage capacity of the Spacetime memories to create queuing structure. **Figure 12** shows a simple input queuing (IQ) structure. A more robust design would use a more sophisticated virtual output queuing (VOQ) to eliminate the head-of-the-line (HOL) blocking problem that an IQ structure creates. While the figure depicts an IQ implementation, a VOQ could be used without loss of generality.

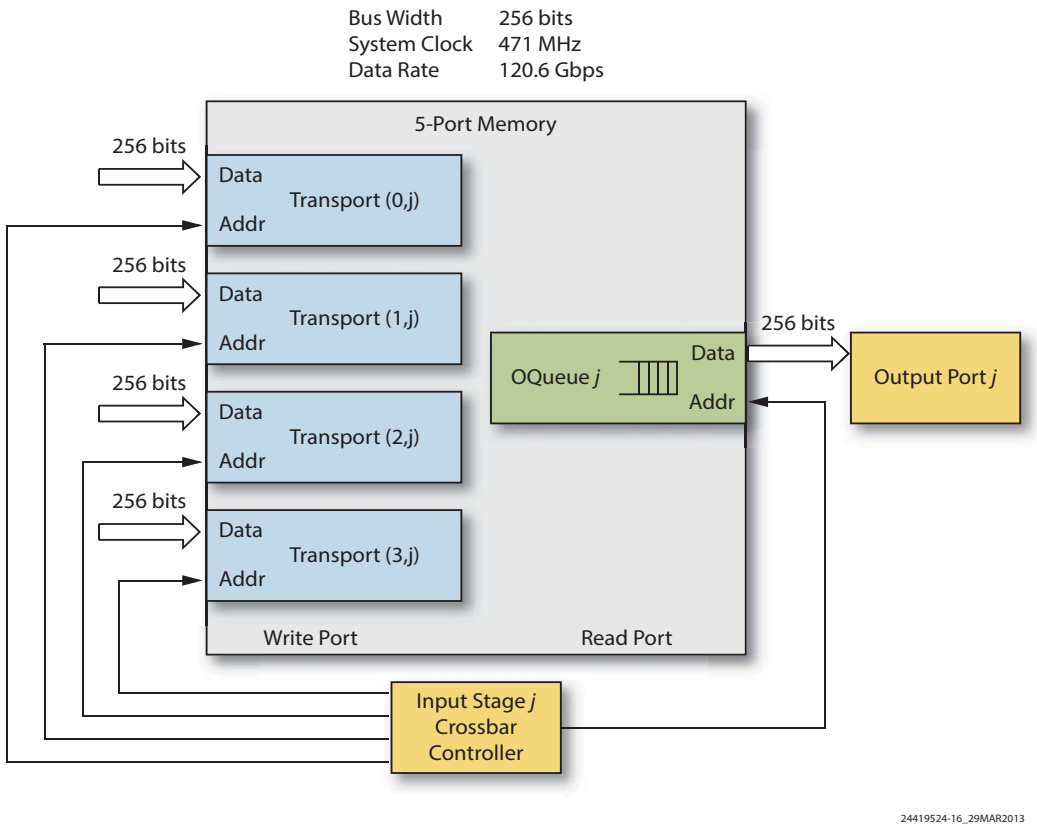
**Figure 12:** ***120G Input Stage Crossbar Element***




Depending on the overall system architecture, the input queues could be implemented with the embedded Spacetime memories, with off-chip DDR3 memory, or not even implemented at all. For rack-mounted equipment, it may be feasible to implement the input queues using Spacetime memory such as that depicted in **Figure 12**. For deep queues and a huge number of flows, use of external DDR3 storage may be required instead of the embedded Spacetime memory. For chassis-based equipment with centralized switching, the input queuing function is typically implemented on the separate line cards instead of the Tabula switch fabric. The key concept is that Tabula crossbar elements can be configured to support integrated queue buffering, if required, based on any system architecture requirements.



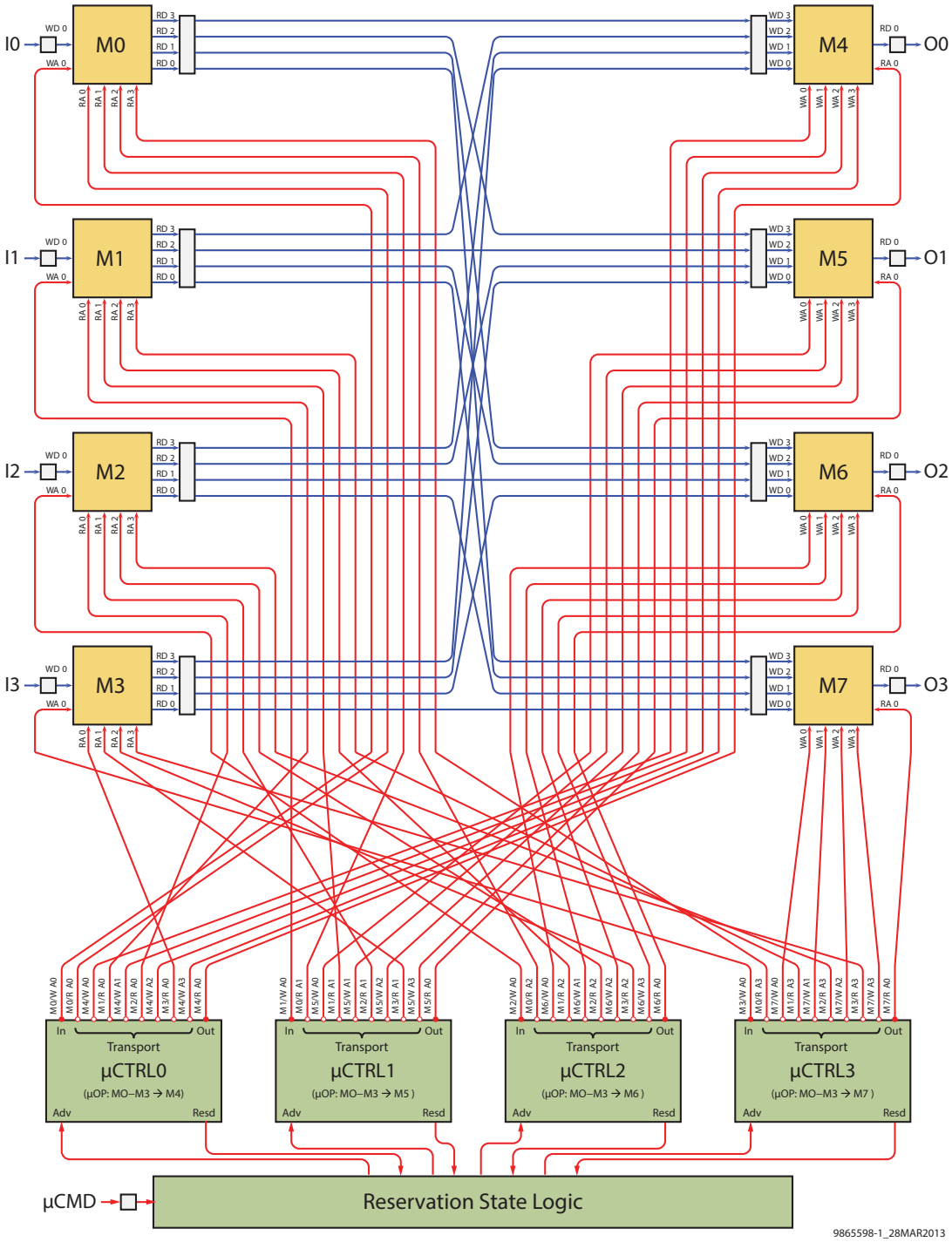
**Figure 13:** ***120G Output Stage Crossbar Element***




Conversely, the output stage crossbar element in **Figure 13** depicts an output queuing structure which may or may not be used depending on the overall system architecture. For a pure input queuing or VOQ architecture, the output queuing structure in **Figure 13** is not required since all queuing is implemented on the ingress side, not the egress side. For pure output queuing or combined input output queuing (CIOQ) systems, the output queuing structure at the output stage could be used.

What's also notable in both **Figure 12** and **Figure 13** is that the switch fabric, as constructed in **Figure 14**, has an inherent 4x fabric speedup. In other words, for any packets written and buffered in the input queue (or groups of VOQs) of **Figure 12**, there are four opportunities for the switch scheduler to read from the queues within one system clock cycle. Conversely, the switch scheduler can use this speedup to resolve output port contention when more than one ingress port wants to send packets to the same egress port. In **Figure 13**, when multiple ingress ports are sending to the same egress port during the same system clock cycle, the output queue simply buffers the 4x speedup incoming packets. A fabric speedup of N, where N is the number of fabric ports, is absolutely essential for output queue architectures. For the 4-port 100G switch in **Figure 12**, a fabric speedup (or folding) of four is required and is provided by the Tabula crossbar elements. Similarly for input queue or VOQ architectures, fabric speedup (usually on the order of 2x) is required or overall switch throughput performance suffers. Again, the required fabric speedup is provided for switching architectures with the Tabula crossbar elements.

**Figure 14:** ***4-Port 120G Crossbar Switch***





With the 2-GHz speed of the Spacetime elements and the multi-ported capabilities of the Spacetime memory, a high-performance data-plane switch fabric can be constructed, capable of transporting 120G data paths. However, the performance of a crossbar switch does not simply depend on the speed of the data-plane switch fabric (denoted in blue in **Figure 14**). The performance also depends on the speed of the control plane (denoted in red in **Figure 14**). Once again, to implement super-fast logic, the Tabula crossbar switch uses 2-GHz Spacetime memory to implement the control-plane logic.

The required minimum speed at which the crossbar controller must be able to configure the switch fabric elements depends on the data path bus width and switching quantum size. For instance, suppose the switching quantum size is chosen to be 512-bits (a typical size for a 64-byte minimum-size packet). With a 256-bit data path bus width, the switching quantum of 512-bits translates to two system clock periods. Thus, the crossbar controller configures the switch fabric every second clock cycle, so that the packet fragments (equal to a minimum-size packet) can traverse the crossbar uninterrupted. The Tabula crossbar controller is designed and optimized to configure the switch fabric every clock cycle at a system clock frequency of 471 MHz; thus, the crossbar controller can handle datapath bus width of 512-bits. So while the ABAX$^2$ device, with its high-performance 3D routing capabilities and super-fast Spacetime memories, is ideally suited to support narrow but fast data buses, the Tabula crossbar controller is designed and optimized to handle very wide data path buses that require a cross-connect configuration setup on every clock cycle.

***Note:*** *The observant reader will ask, "What about the switch fabric speedup required for OQ and VOQ architectures, as mentioned earlier?" The speedup is built into the Spacetime memories using an internal clock at rates up to 2 GHz; so there is no need to double the user's system clock in order to get a 2x fabric speedup, thus simplifying the system design. The input and output stage crossbar elements have the 4x switch fabric speedup built in.*

## Reference Design

A reference design showing how some of the Tabula packet processing building blocks can be used to create a data-plane forwarding device is described below. The design showcases a carrier Ethernet aggregator, connecting 12 subscriber ports operating to a single network port in the uplink path. The subscriber ports operate at 10G rates, and the network port operates at a 100G line rate. Subscriber ports support untagged Ethernet or VLAN (C-tag) packets. Packets at the network ports may have new tags added (S-tag) or be encapsulated with MPLS headers.

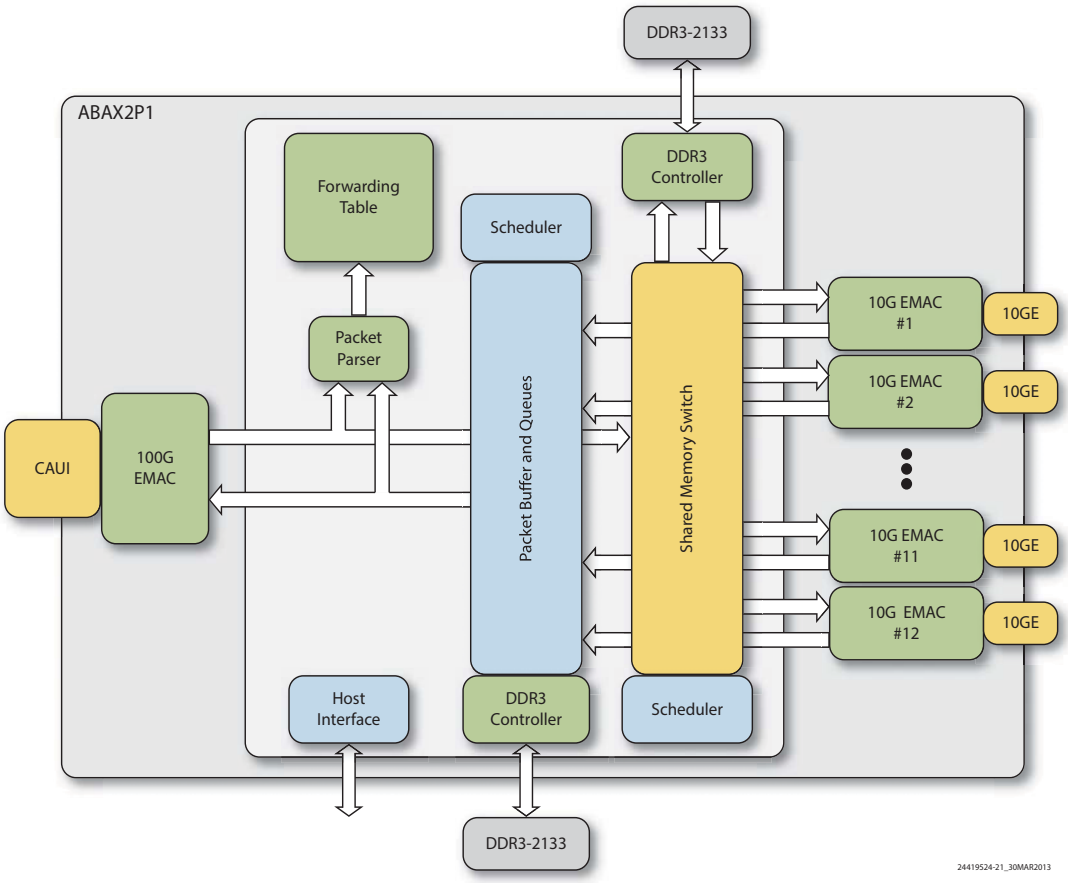
This design should look very familiar to designers that have seen other Tabula reference designs. It combines elements from the 12 x 10G to 100G bridge, and shared memory switch reference designs for an application in the Carrier Ethernet market.

### *12-Port 10G-to-100G Carrier Ethernet Aggregator*

**Figure 15** shows a top-level block diagram of the Ethernet aggregator. The design consists of a single ABAX$^2$P1 3PLD device, and two 4GB DDR3-2133 memory devices for packet buffering. The switching architecture used is a shared memory switch, depicted in yellow.

**Figure 16** and **Figure 17** show the details of the shared memory switch (two figures are used to simplify the depiction). The block diagram in **Figure 16** depicts the packet processing involved in the downstream path, from the 100G network port, to the twelve 10G subscriber ports. The block diagram in **Figure 17** depicts the packet processing involved in the upstream path, from the twelve 10G subscriber ports to the 100G subscriber port.

**Figure 15:** ***12-Port 10G-to-100G Ethernet Aggregator***



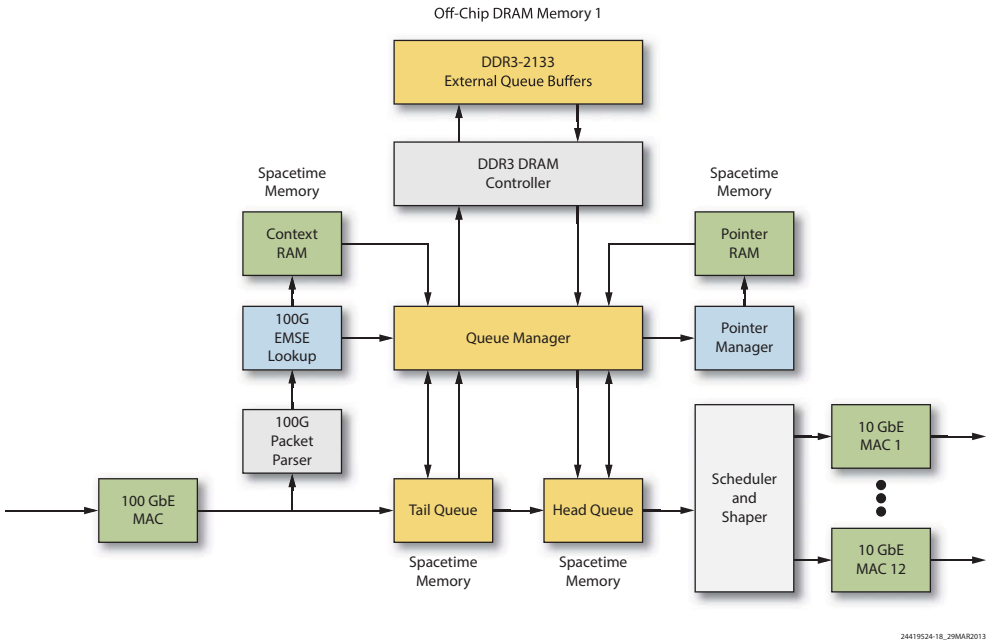

In **Figure 16**, all blocks colored green denote the use of key Tabula packet processing building blocks. Those blocks are the 10/40/100G hard EMACs, packet parser engine, exact-match search engine, embedded Spacetime memory, and the hard DDR3 memory controller. By leveraging the use of these building blocks, the system designer can focus on developing the key determinant of the system performance - the packet queuing and scheduler design.

The block diagram in **Figure 16** illustrates a special design feature: the use of a hybrid packet buffering scheme[6], whereby packets are stored both in internal embedded Spacetime memory, and in external DDR3 DRAM. This scheme is used to address the DRAM latency problem, whereby minimum 64-byte packets at the full 100G line rate can arrive faster than the DRAM's ability to access its own memory banks to store the data (seven whole packets can arrive before the DRAM can store the first packet). With this hybrid packet buffering scheme, any new arriving packet is stored immediately in the tail queue, implemented using on-chip embedded Spacetime memory. Packets continue to accumulate in the tail queue until a certain buffer theshold is reached. Thereafter, any earlier packets waiting in the tail queue are transferred in bulk to the external DDR3 DRAM. With this scheme, the DRAM access time is decoupled from packet arrival times.

6. *S. Iyer, R. R. Kompella, N. McKeown, "Designing Packet Buffers for Router Line Cards", Stanford University Technical Report.*



**Figure 16:** ***100G to 10G***



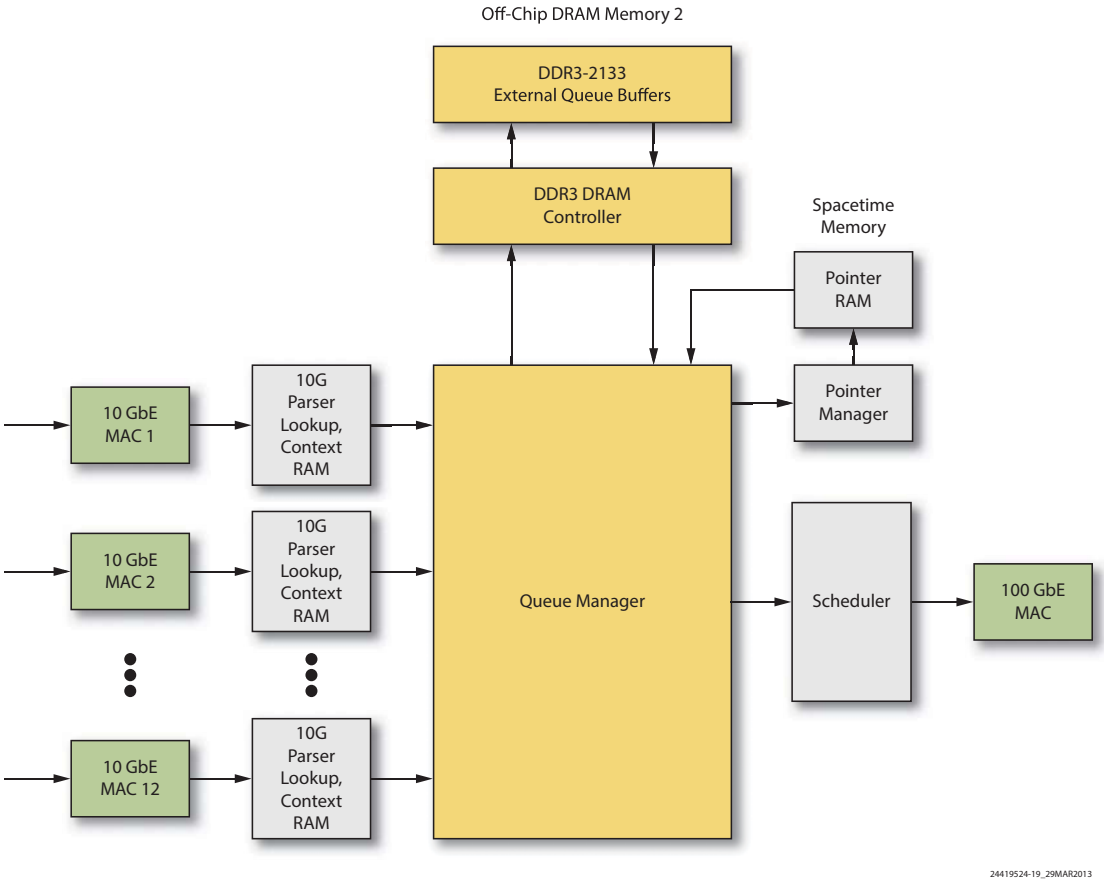

Conversely, the head queue, also implemented using fast Spacetime memory, attempts to maintain a good fill of packets within its buffers. Thus, packets at the head of the queues are instantly available, ready to be scheduled and switched without any long lag in response time as is the case when using DRAM. Whenever the buffer fill in the head queue drops below a certain threshold, a bulk read from DRAM is made, independent of the scheduler's actions.

Other schenarios covered with the hybrid packet buffering scheme are the cases where the total number of buffered packets is low, the queue is near empty, or the queue is completely empty. In those situations, packets flow straight from the tail to the head queue. They do not traverse through the DRAM path at all.

Besides solving the DRAM latency problem, this hybrid packet buffering scheme provides another huge benefit. As described earlier, use of DRAMs for packet storage normally requires an additional 2x memory speedup because of the 65-byte problem. Thus, a memory bandwidth of 400G is required to handle 100G packets (200G to write and 200G to read). Because packets can be stored in the Spacetime memory first and transferred to DRAM only after enough packets have accumulated, the 65-byte problem goes away. In other words, packets can be stored int he tail queue until an integer multiple of 64-bytes have accumulated. Then the DRAM data interface can be utilized with 100% efficiency, eliminating the 65-byte problem. Thus, the memory bandwidth of DRAM 1 in **Figure 16** is 200G instead of 400G.

For **Figure 17**, the block diagram has been simplified slightly to reduce clutter. The blocks labeled "10G Parser, Lookup, Context RAM" is an aggregation of the three blocks shown in **Figure 16** with the labels "100G Packet Parser", "100G EMSE Lookup" and "Context RAM", with data rates operating at 10G instead of 100G. There are 12 instances of such aggregated blocks.

**Figure 17: *10G to 100G***




## Summary

Processing packets at the extremely high rates of 100G, 400G and even 1 Tbps is a daunting challenge. Current devices and technologies are hitting a performance wall, unable to meet these demanding requirements. In addition, the networking landscape is dynamic, with emerging standards like software-defined networking generating a lot of excitement. How do designers handle the technical challenges, which often requires optimization and rigidity, yet remain flexible enough to accomodate designs that are as yet unimaginable as the network evolves and the rate of innovation increases?

What is needed is a device with both high performance and dexterity. What is needed is a device that contains and abundance of optimized, high-performance building blocks that designers can configure to their requirements, and simply drop into their design. With that, a designer is free to focus on the areas where they add the most value to differentiate their products. Tabula is addressing these needs with its ABAX$^2$P1 devices, which are based on revolutionary Spacetime technology and combined with tailored hardware features.



## Authors

***Marc Miller***

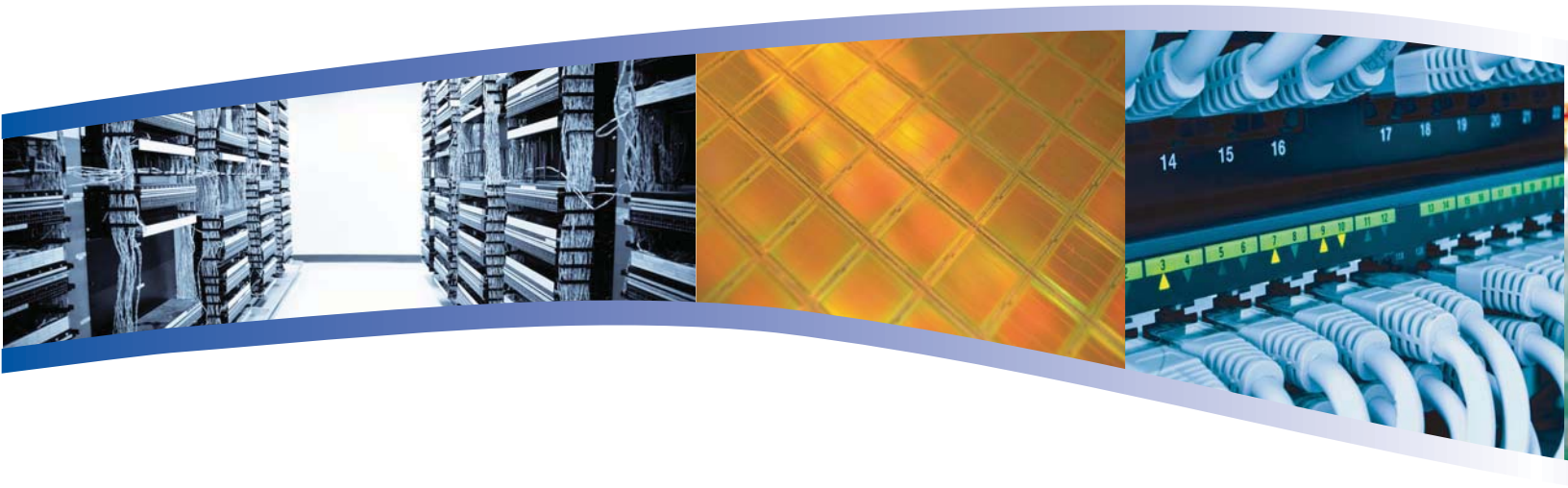
Marc Miller is Sr. Director of Marketing at Tabula where he leads product definition, builds IP partnerships, and creates new business models. Previously at Altera, he led the Stratix GX product definition and drove its transceiver PHY hard IP acquisition that has provided the cornerstone for multiple generations of that product line. At LSI Logic, Marc led the transceiver team that developed the first Gigabit CMOS SerDes and ran the Computer Strategic Business Unit. Marc started his career in microprocessor design at Digital.

***Danny Thom***

Danny Thom is founder and CEO of Wi-Zee and is an industry veteran with over 22 years of hands-on experience in FPGA design and systems engineering in the telecommunications and data networking industry. He has designed and developed industry-leading products working at Cisco, Alcatel-Lucent, Nortel Networks, and Tabula. He received his B.A.Sc. and M.A.Sc. in Electrical Engineering from the University of Waterloo, Canada, and holds four U.S. patents.



| Version | Date | Description of Changes |
|---|---|---|
| V1.0 | March 30, 2013 | Initial release. |



**ABOUT TABULA, INC.**

Tabula is the industry's most innovative programmable logic solutions provider, delivering breakthrough capabilities for today's most challenging systems applications. The company's ABAX$^2$ family of general purpose 3D Programmable Logic Devices (3PLDs), based on Tabula's patented Spacetime architecture and supported by its Stylus compiler, sets a new benchmark for the capability of programmable devices at volume price points, enabling re-programmability not only in FPGA applications but also in those historically served only by ASICs or ASSPs. Headquartered in Santa Clara, California, Tabula has over 130 employees and has assembled a leadership team consisting of industry veterans and successful entrepreneurs. Tabula is backed by top-tier investors with a long-term view toward enduring market leadership. For more information, please visit the Tabula website at www.tabula.com

Tabula, the Tabula logo, ABAX, the ABAX logo, Spacetime, the Spacetime logo, Stylus, the Stylus logo and other designated brands included herein are trademarks of Tabula, Inc. in the United States and other countries. All other trademarks are property of their respective owners

*© 2013 Tabula, Inc. All rights reserved.*

**3250 Olcott St. Santa Clara, CA 95054**

**Phone: 408.986.9140 Fax: 408.986.9146**

**For more information on Tabula, please contact us at sales@tabula.com**

tabula®